



温州肯恩大学
WENZHOUCHEAN UNIVERSITY

CPS 3962 Objected-Oriented Analysis and Design

Final Project 1: Ride-Hailing System

Dr. Hamza Djigal

Assistant Professor

Department of Computer Science, Wenzhou-Kean University

dhamza@kean.edu

March 26, 2026 (Spring)

Contents

1	Project Overview	2
2	Waterfall Methodology Phases	2
2.1	Requirements Gathering	2
2.2	System Analysis	2
2.3	3. System Design	3
2.3.1	Database Schema (Example)	3
2.3.2	UI Wireframes (Placeholders)	4
2.4	Implementation	4
2.5	Testing	4
2.6	Deployment (optional but good as prototype)	5
2.7	7. Documentation & Final Submission	5
3	Optional Advanced Features (Extra Credit)	6

1 Project Overview

Project Title: Ride-Hailing Platform

Objective: Develop a platform connecting riders and drivers, including booking, tracking, payment, and rating functionalities.

Key Features (MVP):

- User Management: Rider and driver registration, login, profile management.
- Booking: Ride request, driver matching, ride history.
- Real-Time Tracking: GPS location tracking of drivers and rides.
- Payments: Fare calculation, payment gateway integration, promo codes.
- Ratings & Feedback: Rider and driver ratings.
- Admin Panel: Monitor rides, users, complaints, and payments.

2 Waterfall Methodology Phases

2.1 Requirements Gathering

- Functional Requirements:
 - Rider: Register, login, book ride, track ride, pay, rate.
 - Driver: Register, login, accept ride, track location, receive payment, rate.
 - Admin: Monitor rides, manage users, generate reports.
- Non-Functional Requirements:
 - Scalability for multiple users.
 - Real-time location tracking.
 - Secure payment processing.
 - Mobile responsiveness.
- Deliverables:
 - Use Case Diagram (Rider, Driver, Admin) etc.
 - Use Case Descriptions (preconditions, main flow, alternate flows) etc.

2.2 System Analysis

Placeholder Diagrams: Students to replace with actual diagrams.

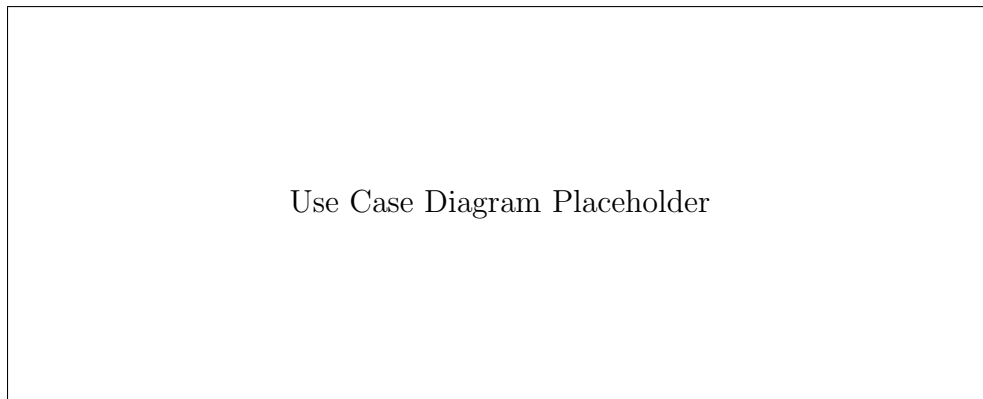


Figure 1: Use Case Diagram (Rider, Driver, Admin)

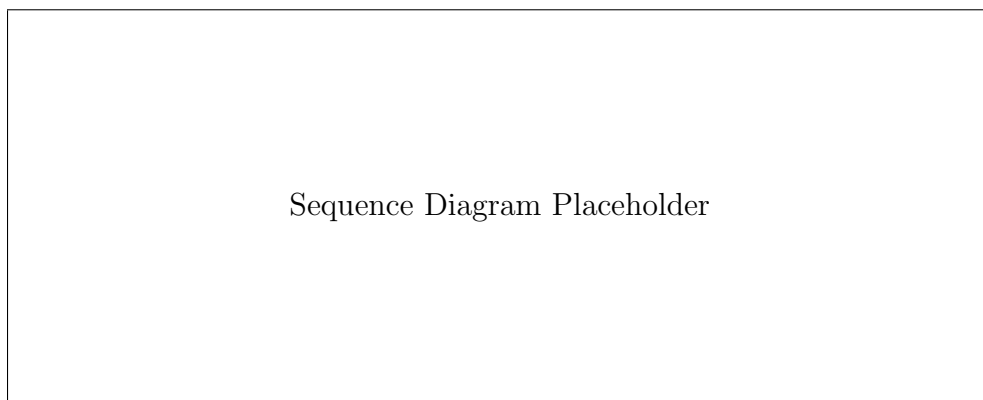


Figure 2: Sequence Diagram: Booking Ride

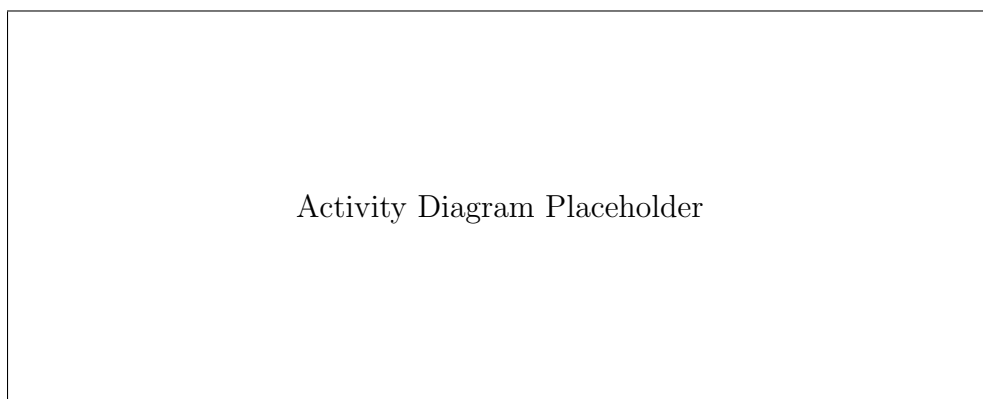


Figure 3: Activity Diagram: Driver Matching Flow

2.3 3. System Design

2.3.1 Database Schema (Example)

Table	Columns	Data Type	Description
Users	user_id (PK), name, email, password, role	INT, VARCHAR	Stores rider/driver info

Rides	ride_id (PK), rider_id (FK), driver_id (FK), status, fare, start_time, end_time	INT, INT, INT, VARCHAR, FLOAT, DATETIME, DATETIME	Ride history
Vehicles	vehicle_id (PK), driver_id (FK), model, license_plate	INT, INT, VARCHAR, VARCHAR	Driver vehicles
Payments	payment_id (PK), ride_id (FK), amount, payment_method, status	INT, INT, FLOAT, VARCHAR, VARCHAR	Payment records
Ratings	rating_id (PK), ride_id (FK), rating, comments	INT, INT, INT, TEXT	Rider/Driver feedback

2.3.2 UI Wireframes (Placeholders)

2.4 Implementation

- Modules:
 1. Authentication & User Management
 2. Ride Booking & Matching
 3. GPS & Tracking
 4. Payment Integration
 5. Ratings & Feedback
 6. Admin Panel
- Follow OO principles: encapsulation, inheritance, polymorphism.
- Use version control (GitHub) for project management.

2.5 Testing

- Unit Testing: Test each class/module.
- Integration Testing: Test booking, payment, and rating flows.
- System Testing: Validate complete application.
- Acceptance Testing: Based on use cases.
- Deliverables:
 - Test Plan document
 - Test Cases with expected vs actual results
 - Bug Tracking Report

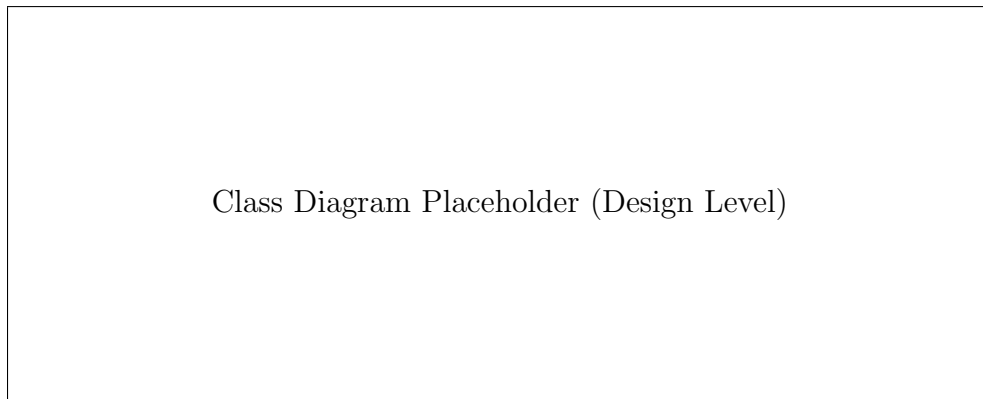


Figure 4: Class Diagram: User, Rider, Driver, Ride, Payment, Vehicle

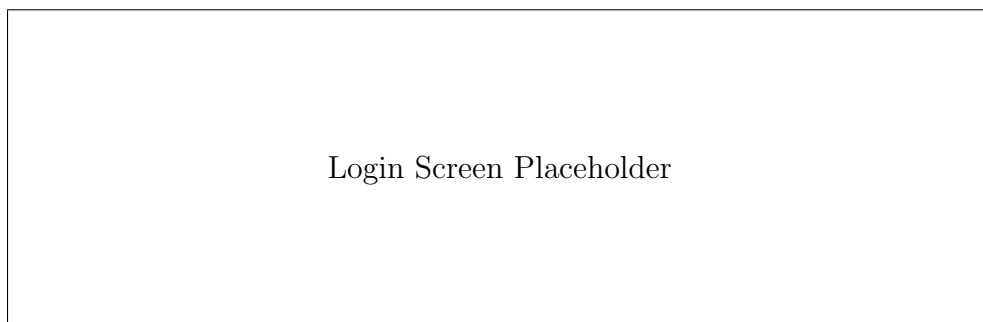


Figure 5: Login Screen

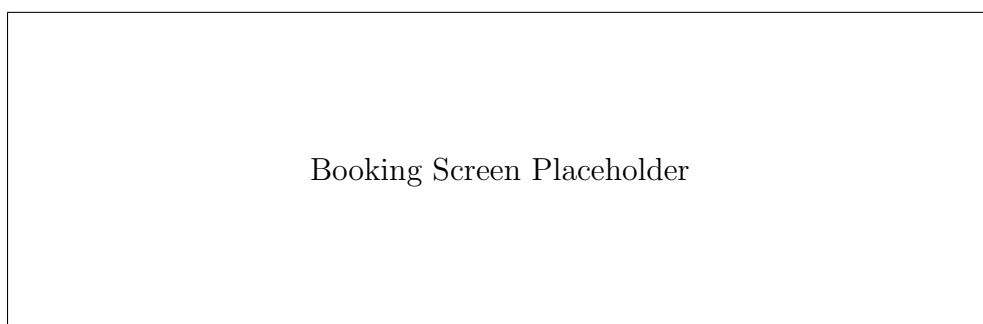


Figure 6: Ride Booking Screen

2.6 Deployment (optional but good as prototype)

- Deploy backend: AWS / Heroku.
- Deploy frontend/mobile app: Web (Netlify), Mobile APK (Android).
- Provide Deployment Guide.

2.7 7. Documentation & Final Submission

- Full OOAD Documentation:
 - Requirements Specification

- UML Diagrams (Use Case, Class, Sequence, Activity)
 - Database Schema
 - UI Screens / Wireframes
 - Test Plan & Results
 - Deployment Guide
- Presentation: demo if possible

3 Optional Advanced Features (Extra Credit)

- Dynamic pricing algorithm
- Multi-language support
- Chat between rider and driver
- Push notifications
- Loyalty & rewards system

Suggested Tech Stack:

- Frontend: ReactJS / Flutter
- Backend: Java Spring Boot / Python Django / Node.js
- Database: PostgreSQL / MySQL
- GPS & Maps: Google Maps API / OpenStreetMap
- Deployment: AWS / Heroku / Docker